

ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
TECHNICAL UNIVERSITY OF CRETE

REPORT ASSIGNMENT 07

NETWORK TRAFFIC MONITORING USING THE SNORT INTRUSION DETECTION SYSTEM

Ασφάλεια Συστημάτων και Υπηρεσιών

Καλανδράνη Όλγα | *2022030130*

Διδάσκων: Σωτήριος Ιωαννίδης

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών, Πολυτεχνείο Κρήτης

ΗΡΥ 413 | Ασφάλεια Συστημάτων και Υπηρεσιών (2025 – 2026)

Part 2:

In this part, custom network traffic packets were created using *Python* and the *Scapy library* inside the given virtual machine.

A T*CP Student Packet* was generated with destination IP 192.168.1.1 and port 54321, which contains the student's name, ID, timestamp. *10 Port San Packets* were created towards 192.168.1.2 for common services (*HTTP, HTTPS, SSH, TELNET, FTP, DNS, RTSP, SQL, RDP, MQTT*). *5 Base64 Malicious Packets* were generated towards 192.168.1.3 on port 8080. *1 UDP DNS Suspicious Domain Packet* was created with the destination set to the VM's DNS server (127.0.0.53 found below) and a query for *malicious.example.com*. Finally, *1 ICMP Ping Test Packet* to 192.168.1.4 with payload *"PingTest-2024"*.

```
student@student-VirtualBox:~/Desktop/assignment7/snort/lab$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search home
```

Figure 1 The DNS IP of the VM

```
student@student-VirtualBox:~/Desktop/assignment7/snort/lab$ python3 PacketsCreation.py
[OK] Created 18 packets and saved to /home/student/Desktop/assignment7/snort/lab/assignme
student@student-VirtualBox:~/Desktop/assignment7/snort/lab$ ls -lh assignment_packets.pca
-rw-rw-r-- 1 student student 1.6K Dec 30 21:41 assignment_packets.pcap
```

Figure 2 Successful creation terminal

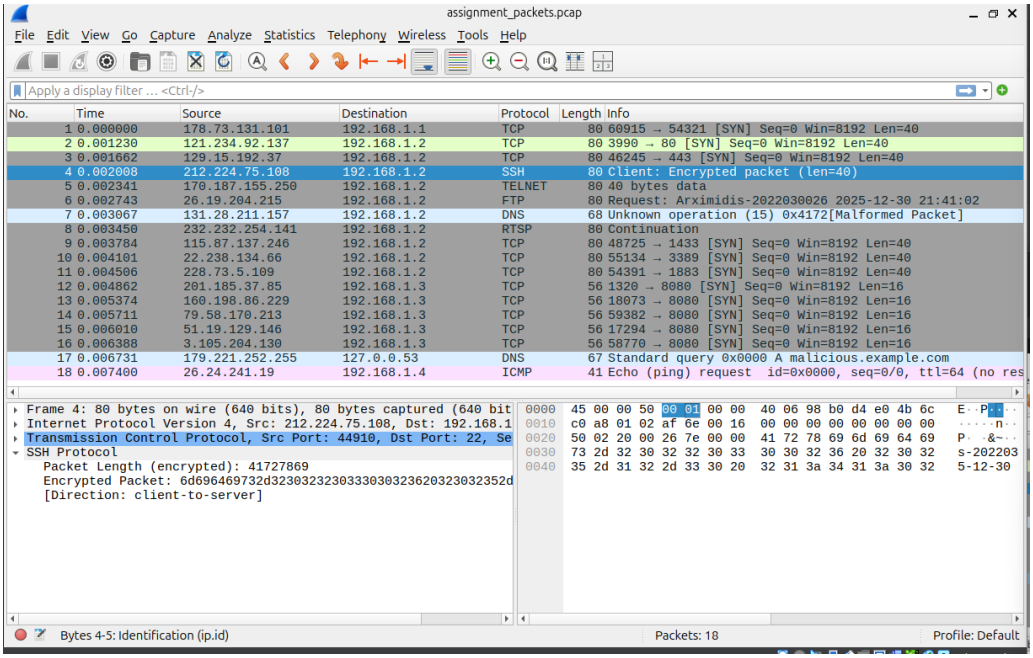


Figure 3 Successful creation PCAP file

Part 3:

In this part of the assignment, custom Snort rules were created in order to detect the packets generated in Part 2. The rules were designed carefully so that only these packets

are detected, whilst avoiding any unnecessary or false alerts. Each rule is based on the specific packet's characteristics, such as protocol, IP address, port, payload.

- Rule for Student Packet:

```
# Student TCP packet
alert tcp any any -> 192.168.1.1 54321 (
    msg:"Student packet detected";
    content:"Arximidis-2022030026";
    sid:1000001;
    rev:1;
)
```

Protocol → TCP
Destination IP → 192.168.1.1
Destination Port → 54321
Source IP and source port are random, so no filtering was applied
Detection is based on the payload content

- Rules for Port Scan Packets (10):

For the port scan , one Snort rule was created for each service. Example for HTTP:

```
# Port scan packets

# HTTP
alert tcp any any -> 192.168.1.2 80 (
    msg:"Port scan HTTP";
    content:"Arximidis-2022030026";
    sid:1000002;
    rev:1;
)
```

Protocol → TCP (UDP only for DNS)
Destination IP → 192.168.1.2
Destination Port → 80 (according to the service)
Source IP and source port are random, so no filtering was applied
Detection is based on the payload content

Similarly Snort rules were created for HTTPS, SSH, TELNET, FTP, DNS, RTSP, SQL, RDP, MQTT

- Rule for Base64 Packets (5):

```
# Base64 malicious packets

alert tcp any any -> 192.168.1.3 8080 (
    msg:"Base64 malicious packet";
    content:"MjAyMjAzMDAyNg==";
    sid:1000012;
    rev:1;
)
```

Protocol → TCP
Destination IP → 192.168.1.3
Destination Port → 8080
Source IP and source port are random, so no filtering was applied
Detection is based on the payload content

- Rule for DNS Suspicious Domain Packet:

```
# DNS suspicious domain packet

alert udp any any -> 127.0.0.53 53 (
    msg:"Suspicious DNS query malicious.example.com";
    content:"malicious.example.com";
    sid:1000013;
    rev:1;
)
```

Protocol → UDP
Destination IP → 127.0.0.53
Destination Port → 53
Source IP and source port are random, so no filtering was applied
Detection is based on the payload content

- Rule for Ping Test Packet:

```
# ICMP Ping test packet

alert icmp any any -> 192.168.1.4 any (
    msg:"CMP PingTest-2024 detected";
    content:"PingTest-2024";
    sid:1000014;
    rev:1;
)
```

Protocol → ICMP
Destination IP → 192.168.1.4
Source IP and source port are random, so no filtering was applied
Detection is based on the payload PingTest-2024

```
##### assignment_packets.pcap #####
        [1:1000001:1] Student packet detected (alerts: 1)
        [1:1000002:1] Port scan HTTP (alerts: 1)
        [1:1000003:1] Port scan HTTPS (alerts: 1)
        [1:1000004:1] Port scan SSH (alerts: 1)
        [1:1000005:1] Port scan TELNET (alerts: 1)
        [1:1000006:1] Port scan FTP (alerts: 1)
        [1:1000007:1] Port scan DNS (alerts: 1)
        [1:1000008:1] Port scan RTSP (alerts: 1)
        [1:1000009:1] Port scan SQL (alerts: 1)
        [1:1000010:1] Port scan RDP (alerts: 1)
        [1:1000011:1] Port scan MQTT (alerts: 1)
        [1:1000012:1] Base64 malicious packet (alerts: 5)
        [1:1000014:1] CMP PingTest-2024 detected (alerts: 1)
#####
```

Figure 4 Snort Rules Testing

Part 4:

In this part of the assignment, the slammer.pcap file was analysed using Wireshark, in order to identify traffic related to the Slammer worm attack. Slammer is a known worm, which targets Microsoft SQL Servers by sending malicious UDP packets.

By analysing the file, we observed that the traffic uses UDP and targets destination port 1434, which is related to the Microsoft SQL Server.

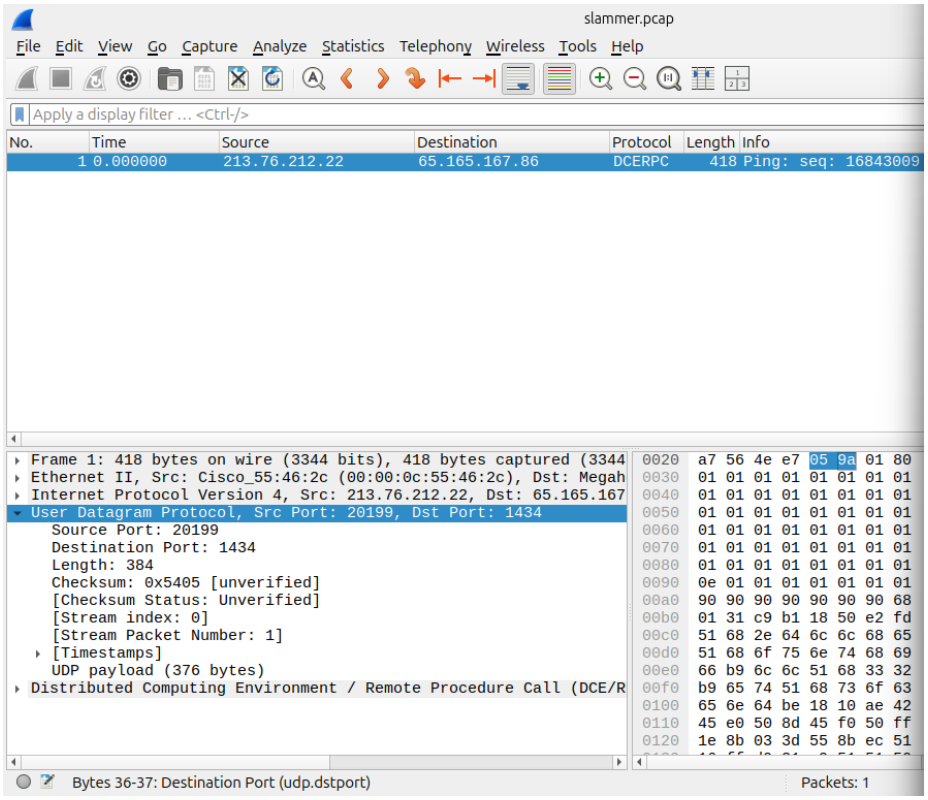


Figure 5 Slammer.pcap file in Wireshark

```
# slammer worm

alert udp any any -> any 1434 (
    msg:"Slammer worm UDP traffic detected";
    content:"|04|";
    sid:2000001;
    rev:1;
)
```

```
##### slammer.pcap #####
        [1:2000001:1] Slammer worm UDP traffic detected (alerts: 1)
#####
```

Figure 6 Slammer worm snort rule

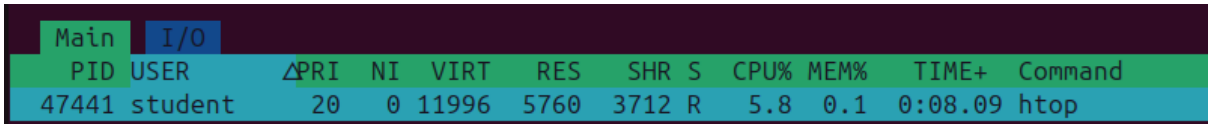
Protocol → UDP
Destination Port → 1434
Source IP and source port are random, so no filtering was applied
Detection is based on a specific byte pattern in the slammer payload

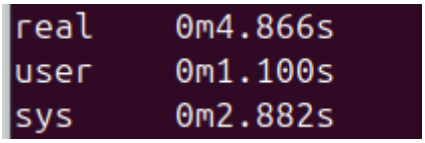
Reference: https://www.giac.org/paper/gsec/3091/ms-sql-slammer-sapphire-worm/105136

Part 5:

In this part of the assignment, a large public PCAP file was used in order to evaluate Snort's performance. The PCAP files were downloaded from this link https://archive.wrccdc.org/pcaps/2024/31/?C=S&O=D and then merged into a large PCAP file (960MB close to 1GB).

Snort was executed on the large PCAP file using the default configuration. The execution time was measured using the time command, and the CPU and memory usage were monitored using the htop tool during the execution. Lastly, no errors or warning were observed during the execution.





In order to achieve lower exec time and reduced CPU load, it is important to use fewer or more targeted Snort rules. Removing unnecessary or similar rules, helps reduce CPU usage and execution time. Additionally, using systems with more CPU cores, can also improve performance, whilst allowing better workload distribution and providing faster analysis.

Part 6:

- Describe how Snort uses rules to detect malicious activity:

  Snort uses a rule-based detection mechanism to identify malicious activity. It analyses network traffic or PCAP files and compares packets against predefined rules. Each snort rule, consists of a header and options, the header defines general criteria such as protocol, IP addresses, ports and traffic direction. The options provide more specific checks, such as matching content or patterns in the packet payload content. When a packet matches a rule, Snort triggers an

action, such as generating an alert. As a result, this allows Snort to detect malicious network activity.

- Mention at least 3 limitations of signature-based intrusion detection systems like Snort with a small description for each one:

    1) Cannot detect unknown attacks → Signature based systems can only detect attacks that already exist in their signature database. New or unknown attacks cannot be detected until a signature is created

    2) Easy to evade by small changes → Attackers can slightly modify their malware or traffic patterns in order to avoid matching existing signatures, whilst making detection harder.

    3) High rate of false positives → If a signature is poorly defined, then normal traffic may be detected as malicious, which results in false alerts.

    4) Performance → As the signature database grows, the system needs more and more time and resources to compare traffic against all signatures, which can eventually reduce performance.

    5) High maintenance → Signature-based systems require frequent updates to keep the database constantly updated, which increases maintenance effort.

    6) Dependency → Detection depends entirely on the accuracy as well as the availability of the signature database. If it is outdated, the system becomes ineffective.

- Pros and cons of using Snort in a real-world scenario:

PROS:

1. Open source, free → Snort is freely available and can be used without any costs.
2. Large community support → A large user community provides documentation, updates and shared rules.
3. Highly customizable → Users can create custom rules specifically for their network needs.
4. Wide protocol support → Snort supports many network protocols and traffic types
5. Easy rule syntax → The rule syntax is simple, straightforward and easy to understand
6. No special hardware required → Snort can run on standard systems without expensive hardware.
7. Cross platform compatibility → Snort can be deployed on multiple operating systems.

CONS:

1. Signature-based limitations → Detection depends on known signatures, making it ineffective against unknown attacks
2. Evasion techniques → Small changes in attack patterns can bypass detection.
3. False positives → Incorrect or poorly written rules may generate false alerts.
4. Performance impact → Large rule sets can reduce performance and increase CPU usage
5. High maintenance → The signature database requires constant updates.
6. Limited visibility to encrypted traffic →Snort cannot inspect encrypted traffic, without additional decryption mechanisms